```
unsigned long throttleLogs(unsigned int  bytesLimit, unsigned int window, const
std::vector<Log> & logs) {
  #  Implement this!
}
```

The STDIN is:
10 bytes,
5 seconds,
4 #there are 4 log lines
0 2 # [0] time=0, 2 bytes;
3 4 # [1] time=3, 4 bytes;
5 5 # [2] time=5, 5 bytes;
7 5 # [3] time=7, 5 bytes;

At time 0, unthrottled, overall total is 2 bytes and current window is 2 bytes.
At time 3, unthrottled, overall total is 6 bytes and current window is 6 bytes.
At time 5, throttled, overall total is 11 bytes more than 10 bytesLimit.
At time 7, unthrottled, as log line time=0 has rolled out of the current window, overall total is 11 bytes and current window is 9 bytes.

Given:

```
struct Log final {
  unsigned int size;  // bytes
  unsigned long timestamp; // seconds since start
}
```

Test case:
10
5
4
0 2
3 4
5 5
7 5

Expected output is 11.

*Note that windowLimit is inclusive.

```
unsigned long throttleLogs(unsigned int bytesLimit, unsigned int window, const
std::vector<Log> &logs) {
  unsigned long totalUnthrottledBytes = 0;
  std::vector<bool> isThrottled(logs.size(), false);

  for (size_t i = 0; i < logs.size(); i++) {
    unsigned long currentTime = logs[i].timestamp;
    unsigned int currentSize = logs[i].size;

    // Calculate the window start time (handle underflow)
    unsigned long windowStart = (currentTime >= window) ? (currentTime - window) : 0;

    // Sum bytes from unthrottled logs within the inclusive window [windowStart,
currentTime]
    unsigned int windowSum = 0;
    for (size_t j = 0; j < i; j++) {
      if (!isThrottled[j] &&
        logs[j].timestamp >= windowStart &&
        logs[j].timestamp <= currentTime) {
        windowSum += logs[j].size;
      }
    }

    // Check if adding current log exceeds the limit
    if (windowSum + currentSize > bytesLimit) {
      isThrottled[i] = true;
    } else {
      totalUnthrottledBytes += currentSize;
    }
  }

  return totalUnthrottledBytes;
}
```